ClariFix programming language

Revised proposal: lexical analyzer submission

COSC 303: Programming Language Design | BSCS 3-3 | Group 4

Revision: lexer-compliance-1. This revision supersedes conflicting lexical rules in the original draft for the lexical analyzer submission. The original draft Markdown and PDF remain historical references. Implementation evidence is recorded separately in `validation.md`.

Group members: Balayanto, Czarina Mae M.; Brase, Mark James B.; Castillo, Marc Lowel J.; Derecho, Jimson Aldrich G.; Hipolito, Brandon Boyd M.; Intong, Sean Gabriel M.; Regencia, Hannah Samantha C.

1. Purpose and implementation phase

ClariFix is a proposed procedural, dynamically typed language intended for beginners. Familiar words such as `set`, `show`, and `ask`, together with errors that explain what happened, why, and how to fix it, support readable programs.

The current deliverable is the lexical analyzer. It reads source text, recognizes lexemes, records locations and identifier identity, produces tokens, and writes an instructor-format Symbol Table. It does not execute programs, enforce full statement grammar, evaluate expressions, infer types, or resolve names. Parsing, semantic analysis, and interpretation are future phases.

Source files use lowercase `.clf`. The analyzer is handwritten in C11 and builds with a desktop GCC or Clang compiler. Python 3 is used only by regression, report-generation, and packaging tools, not by token recognition.

2. Source characters, identifiers, and constants

Files are UTF-8. An optional initial BOM is removed; CRLF and CR line endings normalize to LF. Locations are one-based, and columns count UTF-8 bytes in the normalized source. Code identifiers use ASCII letters, digits, and underscore. Unicode scalar values are allowed inside quoted literals; comment content is ignored by the parser-facing stream and safely escaped in reports.

Identifiers match `[A-Za-z_][A-Za-z0-9_]*`, are case-sensitive, and have at most 32 characters. `score`, `Score`, and `_score1` are distinct valid names. `1score` is invalid. Keywords, Boolean literals, word operators, and noise words are reserved and never classified as IDENTIFIER. Diagnosing a reserved word used in a variable position requires later grammar or semantic analysis, except for lexical noise misuse.

Constant type	Syntax	Valid example
Integer	One or more ASCII digits	`set age = 20.`
Decimal	Digits, dot, digits	`set price = 19.99.`
Character	Exactly one UTF-8 scalar or supported escape inside straight single quotes	`set grade = 'A'.`
String	Zero or more scalars/escapes inside straight double quotes	`set name = "Samantha".`
Boolean	Reserved lowercase literal	`set passed = true.`

These are five scalar literal categories. Arrays are compound syntax, for example `set numbers = [1, 2, 3].`, and are tokenized as brackets, elements, and commas. They are not a sixth scalar literal token.

Minus is a separate operator, so `-20` is MINUS followed by INT. A dot belongs to a decimal only when followed by a digit: `20.` is an integer and a statement terminator; `19.99.` is a decimal and a terminator. Exponents, leading-dot

decimals, and digit separators are not supported. No machine numeric conversion occurs during lexing.

Literals use straight quotes. Supported escapes are backslash followed by n, double quote, single quote, or backslash. Raw newlines, embedded NUL, invalid UTF-8, and unsupported escapes are invalid inside literals. Smart quotes used as literal boundaries are invalid.

3. Fixed vocabulary

There are exactly 57 preloaded fixed spellings: 23 word spellings and 34 symbolic spellings. Quotes are literal boundaries, and comment openers dispatch dedicated recognizers; neither is an additional preloaded symbol entry.

Category	Count	Spellings
Keywords	13	set show ask if elif else while for in function return break continue
Boolean literals	2	true false
Word operators	4	and or not is
Noise words	4	to the of then
Symbol operators	25	+ - * / // % ^ ++ -- && || ! == != < > <= >= = += -= *= /= %= ^=
Delimiters	9	( ) [ ] { } , . :

`range`, `to_int`, `to_decimal`, and `to_text` are identifiers, not reserved keywords. Their interpretation belongs to future phases. There is no implemented `int` declaration keyword: declare variables with `set score = 90.`. The instructor's sample `int` and semicolon illustrate output formatting for another language and do not add those constructs to ClariFix.

4. Operators and delimiters

Arithmetic operators are +, -, *, /, %, ^, ++, and --; // is floor division. Decrement is two ASCII hyphens, never an en dash. The nine operators used to demonstrate Boolean-expression coverage are the three logical operators AND, OR, NOT and six relational operators EQ, NE, LT, GT, LE, GE.

Kind	Spellings	Lexical meaning
Logical	&&, and; ||, or; !, not	Each symbolic/word pair maps to one token kind.
Relational	`== != < > <= >=`	Six comparison token kinds.
Identity word	`is`	IS; `is not` remains IS followed by NOT.
Assignment	`= += -= *= /= %= ^=`	Seven assignment token kinds.
Grouping/indexing	`( ) [ ]`	Recognize and check matching delimiters.
Explicit blocks	`{ }`	Available with `--braces`.
Separators	`, . :`	Comma, simple-statement period, block-header colon.

Longest match applies: ++ precedes +, and <= precedes <. Lexing does not evaluate precedence or validate whether operators form a well-formed expression. Future parser rules must define associativity explicitly; conflicting draft precedence tables are not lexical behavior.

5. Noise words and comments

Noise word	Allowed location	Parser-facing behavior
to	After a `set` target, including completed indexed targets	Emit ASSIGN retaining original lexeme `to`.
then	After an `if`/`elif` condition, immediately before the next significant colon	Omit filler token.
the, of	Immediately after `set`/`show`/`ask`, or after a `set` assignment separator	Omit filler token. Multiple fillers at these boundaries are allowed.

The report retains each allowed noise occurrence as NOISE_WORD. Misplaced noise produces E105. Full expression and statement validity remain parser checks.

Line comments start with # and end before the physical newline or EOF. Block comments use `/* ... */`, may span lines, and do not nest. Both are absent from parser tokens and present in the submission report as LINE_COMMENT or BLOCK_COMMENT. An unclosed block comment produces E106 at its opener and has no successful report row. `//` is always floor division, never a comment marker.

6. Layout modes

Default mode follows the indentation grammar. A new indentation level adds exactly four spaces; a dedent returns to a previous level. Code indentation cannot contain tabs. Blank and comment-only lines do not change indentation. Nonempty logical lines produce NEWLINE; block changes produce INDENT/DEDENT. Parentheses and square brackets suppress layout while open. Remaining logical newline and dedents are emitted at EOF.

Comments behave as whitespace; a newline inside a block comment can end preceding code outside grouping. A multiline expression in default mode must be enclosed in parentheses or square brackets. Spaces and tabs between tokens are separators. Braces are invalid in default mode.

Explicit braces mode is selected with `--braces`. In that mode, braces delimit blocks and all layout tokens are suppressed. Spaces, tabs, and newlines separate tokens; periods terminate simple statements. This is an explicitly selected alternate lexical mode, not an automatic mixture with indentation syntax. The lexer recognizes braces and checks matching pairs; a future parser must validate complete brace-mode statements.

Default example:

```
set the age to 20.
if age >= 18 then:
    show "Adult".
else:
    show "Minor".
```

Braces example, requiring `--braces`:

```
set age = 20.
if (age >= 18) { show "Adult". } else { show "Minor". }
```

7. Finite automata and symbol identity

Tokenization uses handwritten character transitions. The fixed vocabulary initializes a sparse trie: a state contains outgoing character edges and an optional accepting symbol ID. Words advance one character at a time, accept a fixed type only at the entire word boundary, and otherwise become identifiers. Thus `ifx` remains an identifier rather than IF followed by another token. Operator recognition remembers the last accepting state to implement longest match.

The trie also stores character paths for dynamic identifier identity. Terminal nodes carry stable symbol IDs. Repeated spellings reach the same node without whole-word comparisons; different spellings cannot merge because of a hash collision. This replaces the draft's proposed hash table. Each symbol stores original spelling, token kind, fixed/identifier origin, first location, and occurrence count. The 57 fixed entries initialize each independent lexing run; new identifiers append as encountered.

Number, literal, UTF-8, and comment recognizers use character classes, states, and limited lookahead. Indentation and delimiter stacks are separate auxiliary logic and are not described as finite automata. See `lexer-automata.md` for the transition tables and accepting actions.

No premade lexical analyzer, word-level matching, or completed-lexeme string comparison is used for source token classification or identifier identity. CLI option/extension comparisons in `main.c` are unrelated to source tokenization. Storage copying and length measurement do not classify words.

8. Symbol Table and developer outputs

The instructor-format Symbol Table lists successfully recognized source lexemes in occurrence order, retaining repeated names, literals, comments, and allowed noise words. It is generated with `--report FILE`. It has exactly two tab-separated columns:

```
TOKEN TYPE    LEXEME
SET           set
IDENTIFIER    age
NOISE_WORD    to
INT_LITERAL   20
PERIOD        .
```

The spacing above illustrates columns; actual files use one tab. The example corresponds to `set age to 20.`. Internal IDENT/INT/DECIMAL/CHAR/STRING/BOOL/NOISE map to IDENTIFIER/INT_LITERAL/DECIMAL_LITERAL/CHAR_LITERAL/STRING_LITERAL/BOOL_LITERAL/NOISE_WORD. Other recognized token names remain unchanged. Synthetic EOF/NEWLINE/INDENT/DEDENT, invalid fragments, and unused preloaded entries are excluded.

Original literal quotes and comment markers remain present. Backslash, tab, LF, and CR escape as `\\`, `\t`, `\n`, and `\r`; other controls, DEL, and invalid bytes use `\xHH`. Valid UTF-8 is preserved. Each lexeme occupies one physical report row. `report-format.md` specifies the full name mapping and row semantics.

Developer exports remain separate: `--tokens FILE` writes detailed parser-facing tokens with locations and symbol IDs, and `--symbols FILE` writes the internal deduplicated symbol inventory. Tokens go to stdout when `--tokens` is absent. Output files must not already exist; exclusive creation protects source files and previous results.

9. Errors and recovery

Code	Lexical condition
E100	Source filename does not end in lowercase `.clf`.
E102	Unclosed literal or unmatched/unclosed bracket.
E103	Invalid code character, invalid literal encoding, NUL in a literal, or brace in indentation mode.
E104	Identifier exceeds 32 characters.
E105	Noise word appears in an unsupported position.
E106	Unclosed block comment.
E107	Tab in default-mode code indentation.
E108	Invalid indentation increment or dedent.
E109	Smart quote used as a literal boundary.
E110	Number touches a name, including a name starting with digits.
E111	Unsupported literal escape.
E112	Character literal does not hold exactly one scalar/escape.

Diagnostics state where the problem occurred, what happened, why, and how to fix it. Invalid bytes/control characters are escaped so diagnostics remain valid UTF-8. Recovery skips the damaged physical line, discards its open bracket entries, and continues where possible. After five errors, scanning stops with `Fix these first, then run again`.

Exit codes are 0 for lexical success, 1 for lexical errors, and 2 for usage, memory, or I/O failures. On errors, token/report output is partial and must not be treated as executable input. E101 (missing colon), E113 (missing period), E202 (reserved word used as a variable), undefined names, and type/runtime errors require later phases. Lexical success does not certify a runnable program.

10. Build, verification, and submission

Windows, with desktop GCC and Python on PATH:

```
.\build.ps1 -Test
.\build.ps1 -Examples -OutputDirectory build\fresh-reports
.\build\clarifix.exe --report build\fresh-average.txt examples\average.clf
.\build\clarifix.exe --braces examples\braces.clf
```

Choose fresh report filenames/directories for every run. `-Compiler` accepts an explicit GCC/Clang path if needed. Direct compilation and Linux/macOS commands appear in README.

Regression tests cover exact tokens and locations, identifier identity and keyword prefixes, constants, operators, comments, Unicode, noise normalization, layout modes, invalid bytes, report format, recovery, and safe output creation. `average.clf`, `features.clf`, `braces.clf`, and `checklist.clf` must succeed; `errors.clf` must fail with exactly five intentional lexical errors. `validation.md` records actual runs and package checks.

Submit source, build instructions, tests, examples, report samples, automata documentation, validation evidence, and this revised proposal in printable PDF form. The group must supply its actual GitHub repository URL, submit that link and the ZIP through the course's central repository, confirm the calendar deadline, and bring its index card and printed proposal. The screenshot states Thursday 11:59 PM but supplies no date or timezone.

References and provenance

- Group 4, original ClariFix draft proposal, preserved in the project root as historical Markdown and PDF.

- Instructor, COSC 303 Lexical Analyzer Guidelines & Requirements, supplied screenshot.

- Aho, Lam, Sethi, and Ullman, Compilers: Principles, Techniques, and Tools, for lexical analysis and finite automata concepts.

- Sebesta, Concepts of Programming Languages, for language-design background.

This revision describes the current lexical deliverable and explicitly marks future phases. It does not claim that the proposed interpreter or course hand-in has been completed.